



UNIVERSITAT
POLITÈCNICA
DE VALÈNCIA



Master Universitario en Inteligencia Artificial, Reconocimiento de Formas e Imagen Digital
Universidad Politécnica de Valencia

Entrenamiento de Redes Neuronales: MNIST y CIFAR-10

TRABAJO REDES NUERONALES ARTIFICIALES

Máster Universitario en Inteligencia Artificial, Reconocimiento de Formas e Imagen Digital

Gómez Corral, Miquel

CAPÍTULO 1

Introducción

Este trabajo tiene como objetivo la creación y entrenamiento desde cero de modelos de redes neuronales para la resolución de dos tareas de clasificación de imágenes: MNIST y CIFAR-10. Se usarán diferentes técnicas y arquitecturas para cada uno de los datasets, y se evaluará el rendimiento de los modelos entrenados.

Hardware utilizado

Los experimentos se han realizado en mi máquina personal: GPU NVIDIA RTX 5070 (12 GB VRAM), procesador AMD Ryzen 9 9000X, 64 GB de RAM y Ubuntu 24.04 LTS.

Todos los modelos han podido ser entrenados y evaluados sin problemas.

1.1 Código entregado

Se ha entregado un repositorio con todo el código necesario para reproducir los experimentos realizados en este trabajo. El repositorio tiene como punto de entrada el archivo auto-documentado `app/main.py`. Con él, se pueden entrenar los modelos para ambas tareas y ajustar fácilmente los hiperparámetros con argumentos.

El resto de la estructura del repositorio se encuentra en el archivo `README.md` del repositorio. Ahí se explica también como instalar las dependencias necesarias y cómo ejecutar el código.

Para lanzar los dos experimentos que han conseguido los mejores resultados, se pueden usar los siguientes comandos.

```
python ./main.py mnist -des "Best experiment" -b 1024
python ./main.py cifar -des "Best experiment" -b 256
```

Ajustar el batch size (-b) según la memoria disponible en la GPU. Con 12 GB de VRAM, estos valores funcionan correctamente.

CAPÍTULO 2

Datasets

2.1 Ejercicio 1: MNIST

El dataset de MNIST es un conjunto de datos muy conocido en el ámbito del aprendizaje automático y la visión por computadora. Consiste en imágenes de dígitos manuscritos (del 0 al 9) en escala de grises, con un tamaño de 28x28 píxeles. El conjunto de datos contiene un total de 70,000 imágenes, divididas en 60,000 imágenes para entrenamiento y 10,000 imágenes para prueba.

La red utilizada para este ejercicio es una 'Multilayer Perceptron' (MLP) con 4 capas ocultas, una skip layer personalizada, que usa BatchNorm1d entre capas, Dropout del 45 % y ReLU como función de activación.

Para su entrenamiento, se ha utilizado el optimizador AdamW, OneCycleLR como scheduler y CrossEntropyLoss con label smoothing de 0.1 como función de pérdida. El modelo se ha entrenado durante 200 épocas con un tamaño de lote de 1024, y se ha obtenido una accuracy del 99,47 %.

A continuación, se muestran los resultados que se han obtenido durante la implementación de las técnicas hasta llegar al objetivo.

La configuración base ha sido la siguiente: Adam, CrossEntropyLoss, Learning rate decay, data augmentation con RandomAffine (degrees=15, translate=(0.1, 0.1), scale=(0.9, 1.1), shear=10), arquitectura MLP con capas: [784, 1024, 1024, 512, 10]

Técnica	Accuracy %	Error %
Base (MLP 784-1024-1024-512)	97.57	2.43
Data augmentation adaptativo	98.36	1.64
Skip connections	98.74	1.26
Red más densa (784-1536-1536-512-256)	99.13	0.87
Skip connections adicionales	99.20	0.80
Label smoothing y scheduler	99.38	0.62
Ajuste final de hiperparámetros	99.47	0.53

Tabla 2.1: Resultados de accuracy para MNIST de las distintas técnicas aplicadas.

Al final, el uso de una red neuronal más densa con skip connections, junto con técnicas de regularización como label smoothing, data augmentation, y un ajuste fino de hiperparámetros, ha permitido alcanzar una accuracy del 99,47 % en el conjunto de prueba, lo que nos deja por debajo del 0,6 % de error.

2.2 Ejercicio 2: CIFAR-10

El dataset CIFAR-10 es otro conjunto de datos ampliamente utilizado. Consiste en 60,000 imágenes a color (3 canales, RGB) de 32x32 píxeles, divididas en 10 clases diferentes (avión, automóvil, pájaro, gato, ciervo, perro, rana, caballo, barco y camión). El conjunto de datos se divide en 50,000 imágenes para entrenamiento y 10,000 imágenes para prueba.

A diferencia del ejercicio anterior, la red utilizada sí es convolucional. En concreto, se compone de 3 bloques convolucionales de la clase 'ResNet' provista. Esta capas de 64, 128 y 256 canales, están seguidas de una capa fully connected al final. Se usa BatchNorm2d y ReLUs entre capas convolucionales, mientras que las fully conectadas tienen un BatchNorm1d. La primera FF también tiene un Dropout del 50 % y ReLU como función de activación. La última capa de la convolución se aplanada y se conecta a una capa fully connected que produce las 10 salidas correspondientes a las clases del dataset.

Para su entrenamiento, se ha vuelto a usar el optimizador AdamW y CrossEntropyLoss con label smoothing de 0.1 como función de pérdida, pero ahora con CosineAnnealingLR como scheduler. El modelo se ha entrenado durante 100 épocas con un tamaño de lote de 256, y se ha obtenido una accuracy del 93,44 %.

La configuración base ha sido la siguiente: AdamW, CrossEntropyLoss (smooth 0.1), OneCycleLR, RandomHorizontalFlip, RandomAffine(degrees=15, translate=(0.1, 0.1), scale=(0.9, 1.1), shear=10), Canales convoluciones: [3, 64, 128, 256]

Técnica	Accuracy %	Error %
Base	84.72	15.28
Rand augment con flip y Crop	86.41	13.59
Resnet	88.17	11.83
Dropout	88.66	11.34
Ruido a las imágenes (no funciona)	78.12	21.88
CosineAnnealingLR	90.50	9.50
Mejores hiperparámetros y más Epochs	91.81	8.19
CNN Más densa (3-32-64-128-256-512)	93.44	6.56

Tabla 2.2: Resultados de accuracy para CIFAR-10 de las distintas técnicas aplicadas.

Al final, el uso de una red convolucional más densa, junto con las técnicas de data augmentation y un buen ajuste de hiperparámetros, ha permitido alcanzar la accuracy objetivo del 93,44 % en el conjunto de prueba, lo que nos deja por debajo del 8 % de error.